

Name: A.ADARSH

Reg no:192365002

Simulation output

```
<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>SR-MACS | Swarm Robotics Multi-Agent Coordination System</title>

<style>

:root{

  --bg:#060a14; --bg2:#0a1122; --card:rgba(255,255,255,0.045); --card-
border:rgba(255,255,255,0.09);

  --cyan:#00e5ff; --blue:#3b82f6; --green:#22c55e; --orange:#ff9800; --red:#ff3b3b; --
gold:#ffd54a;

  --text:#e7f1ff; --dim:#8ca0c2; --dim2:#5f7291;

}

*{box-sizing:border-box;}

html,body{margin:0;padding:0;background:var(--bg);color:var(--text);font-family:'Segoe
UI',Roboto,Arial,sans-serif;}

body{background:radial-gradient(circle at 20% 0%, #0d1830 0%, #060a14 55%, #04060c
100%);min-height:100vh;}

a{color:var(--cyan);}

::-webkit-scrollbar{width:8px;height:8px;}

::-webkit-scrollbar-thumb{background:rgba(255,255,255,0.15);border-radius:4px;}

.glass{background:var(--card);border:1px solid var(--card-border);border-
radius:14px;backdrop-filter:blur(10px);box-shadow:0 8px 24px rgba(0,0,0,0.35);}

.app{max-width:1600px;margin:0 auto;padding:14px 16px 40px;}

/* ===== TOP BAR ===== */

.topbar{display:flex;align-items:center;justify-content:space-between;padding:10px
18px;margin-bottom:12px;}

.brand{display:flex;flex-direction:column;line-height:1.1;}
```

```

.brand .logo{font-weight:800;font-size:20px;letter-spacing:1px;color:var(--cyan);text-
shadow:0 0 18px rgba(0,229,255,0.5);}

.brand .subb{font-size:10.5px;color:var(--dim);letter-spacing:0.5px;}

.center-title{text-align:center;flex:1;}

.center-title h1{margin:0;font-size:26px;letter-spacing:3px;font-
weight:800;background:linear-gradient(90deg,#00e5ff,#3b82f6);-webkit-background-
clip:text;background-clip:text;color:transparent;}

.center-title .sub{color:var(--dim);font-size:12px;letter-spacing:2px;margin-top:2px;}

.status-badge{padding:8px 14px;border-radius:20px;font-size:12.5px;font-
weight:700;background:rgba(34,197,94,0.12);border:1px solid
rgba(34,197,94,0.4);color:#7CFFB2;white-space:nowrap;transition:all .3s;}

.status-badge.paused{background:rgba(255,152,0,0.12);border-
color:rgba(255,152,0,0.4);color:#ffcc80;}

.status-badge.emergency{background:rgba(255,59,59,0.15);border-
color:rgba(255,59,59,0.5);color:#ff8a8a;animation:blinkBadge 1s infinite;}

@keyframes blinkBadge{0%,100%{opacity:1;}50%{opacity:0.45;}}

/* ===== TITLE BAR / CONTROLS ===== */

.titlebar{padding:16px 20px;margin-bottom:14px;text-align:center;}

.titlebar h2{margin:0 0 4px;font-size:19px;color:#fff;font-weight:700;}

.titlebar .small-sub{color:var(--dim);font-size:12.5px;margin-bottom:14px;}

.ctrl-buttons{display:flex;gap:10px;justify-content:center;flex-wrap:wrap;}

button{cursor:pointer;border:none;outline:none;font-family:inherit;}

.btn{padding:10px 18px;border-radius:10px;font-size:12.5px;font-weight:700;letter-
spacing:0.5px;transition:transform .15s, box-shadow .15s;color:#fff;}

.btn:hover{transform:translateY(-2px);}

.btn:active{transform:translateY(0);}

.btn-start{background:linear-gradient(135deg,#22c55e,#0ea55f);box-shadow:0 4px 14px
rgba(34,197,94,0.35);}

.btn-pause{background:linear-gradient(135deg,#ffb300,#ff8f00);box-shadow:0 4px 14px
rgba(255,152,0,0.3);}

.btn-reset{background:linear-gradient(135deg,#3b82f6,#1d4ed8);box-shadow:0 4px 14px
rgba(59,130,246,0.3);}

.btn-random{background:linear-gradient(135deg,#8b5cf6,#6d28d9);box-shadow:0 4px
14px rgba(139,92,246,0.3);}

.btn-emergency{background:linear-gradient(135deg,#ff3b3b,#b71c1c);box-shadow:0 4px
14px rgba(255,59,59,0.4);}

```

```
.btn-export{background:linear-gradient(135deg,#00bcd4,#0097a7);box-shadow:0 4px 14px
rgba(0,188,212,0.3);}
```

```
.btn:disabled{opacity:0.4;cursor:not-allowed;transform:none;}
```

```
.emergency-banner{display:none;background:linear-
gradient(90deg,#7a0000,#b30000);color:#fff;text-align:center;padding:10px;border-
radius:10px;font-weight:800;letter-spacing:1px;margin:0 0 14px;animation:blinkBadge 0.8s
infinite;}
```

```
.emergency-banner.show{display:block;}
```

```
/* ===== LAYOUT ===== */
```

```
.main-layout{display:grid;grid-template-columns:290px 1fr 350px;gap:14px;align-
items:start;}
```

```
@media (max-width:1250px){.main-layout{grid-template-columns:1fr;}}
```

```
.panel{padding:14px 16px;margin-bottom:14px;}
```

```
.panel h3{margin:0 0 10px;font-size:13px;letter-spacing:1px;color:var(--cyan);text-
transform:uppercase;border-bottom:1px solid var(--card-border);padding-bottom:8px;}
```

```
.field{margin-bottom:14px;}
```

```
.field label{display:flex;justify-content:space-between;font-size:12px;color:var(--
dim);margin-bottom:6px;}
```

```
.field label span.val{color:#fff;font-weight:700;}
```

```
input[type=range]{width:100%;height:5px;border-
radius:4px;background:rgba(255,255,255,0.12);outline:none;-webkit-appearance:none;}
```

```
input[type=range]::-webkit-slider-thumb{-webkit-
appearance:none;width:15px;height:15px;border-radius:50%;background:var(--cyan);box-
shadow:0 0 8px rgba(0,229,255,0.7);cursor:pointer;}
```

```
select{width:100%;padding:8px 10px;border-
radius:8px;background:#0d162c;color:#fff;border:1px solid var(--card-border);font-
size:12.5px;}
```

```
.algo-desc{margin-top:10px;padding:10px
12px;background:rgba(0,229,255,0.06);border:1px solid rgba(0,229,255,0.2);border-
radius:10px;font-size:11.5px;color:var(--dim);}
```

```
.algo-desc b{color:var(--cyan);display:block;margin-bottom:3px;font-size:12.5px;}
```

```
.indicators{display:grid;grid-template-columns:1fr 1fr;gap:8px;font-size:11.5px;color:var(--
dim);}
```

```
.indicators div{display:flex;align-items:center;gap:6px;}
```

```

/* ===== SIMULATION AREA ===== */

.sim-wrap{position:relative;}

#simCanvas{width:100%;height:auto;aspect-ratio:1100/620;display:block;border-
radius:16px;background:linear-gradient(160deg,#060c1a,#0a1428);border:1px solid var(--
card-border);box-shadow:0 10px 40px rgba(0,0,0,0.5);cursor:crosshair;}

.help-box{position:absolute;bottom:12px;left:12px;padding:10px 12px;font-
size:11px;color:var(--dim);max-width:230px;line-height:1.6;}

.help-box b{color:var(--cyan);display:block;margin-bottom:4px;font-size:11.5px;}

.details-panel{position:absolute;top:12px;right:12px;width:230px;padding:12px
14px;display:none;}

.details-panel.hidden{display:none;}

.details-panel:not(.hidden){display:block;}

.panel-head{display:flex;justify-content:space-between;align-items:center;margin-
bottom:8px;border-bottom:1px solid var(--card-border);padding-bottom:6px;}

.panel-head strong{color:var(--cyan);font-size:13px;}

.close-btn{background:none;color:var(--dim);font-size:14px;padding:2px 6px;border-
radius:6px;}

.close-btn:hover{color:#fff;background:rgba(255,255,255,0.08);}

.detail-row{display:flex;justify-content:space-between;font-size:11.5px;color:var(--
dim);margin-bottom:6px;}

.detail-row b{color:#fff;}

.mini-status{position:absolute;top:12px;left:12px;display:flex;gap:8px;flex-wrap:wrap;max-
width:60%;}

.mini-status span{padding:5px 10px;border-radius:8px;font-size:10.5px;font-
weight:700;background:rgba(0,0,0,0.4);border:1px solid var(--card-border);}

/* ===== RIGHT PANEL ===== */

.metrics-grid{display:grid;grid-template-columns:1fr 1fr;gap:10px;}

.metric-card{padding:12px;border-
radius:12px;background:rgba(255,255,255,0.04);border:1px solid var(--card-border);text-
align:center;}

.metric-card .mv{font-size:20px;font-weight:800;color:#fff;}

.metric-card .ml{font-size:10px;color:var(--dim);letter-spacing:0.5px;margin-top:2px;text-
transform:uppercase;}

.metric-card.cyan .mv{color:var(--cyan);}

```

```

.metric-card.green .mv{color:var(--green);}
.metric-card.orange .mv{color:var(--orange);}
.metric-card.red .mv{color:var(--red);}

.row-stats{display:flex;justify-content:space-between;font-size:12px;color:var(--dim);margin-bottom:8px;}
.row-stats b{color:#fff;}

table{width:100%;border-collapse:collapse;font-size:11px;}
th{text-align:left;color:var(--dim);font-weight:600;padding:6px 4px;border-bottom:1px solid var(--card-border);text-transform:uppercase;font-size:9.5px;}
td{padding:6px 4px;border-bottom:1px solid rgba(255,255,255,0.04);color:#dce8fb;}
.table-scroll{max-height:220px;overflow-y:auto;}
.badge{padding:3px 8px;border-radius:6px;font-size:9.5px;font-weight:700;}
.badge.ok{background:rgba(34,197,94,0.15);color:#7CFFB2;}
.badge.warn{background:rgba(255,152,0,0.15);color:#ffcc80;}
.badge.danger{background:rgba(255,59,59,0.18);color:#ff9a9a;}
.badge.off{background:rgba(255,255,255,0.08);color:var(--dim);}
.tag{padding:3px 7px;border-radius:6px;font-size:9.5px;font-weight:700;}
.tag.moving{background:rgba(59,130,246,0.15);color:#a9c8ff;}
.tag.arriving{background:rgba(34,197,94,0.15);color:#7CFFB2;}
.tag.idle{background:rgba(255,255,255,0.08);color:var(--dim);}
.batt-low{color:#ff8a8a !important;}
.batt-mid{color:#ffcc80 !important;}
.batt-ok{color:#7CFFB2 !important;}

.chart-box{margin-bottom:10px;}
.chart-box .clabel{font-size:10.5px;color:var(--dim);margin-bottom:4px;text-transform:uppercase;letter-spacing:0.5px;}
canvas.chart{width:100%;height:70px;border-radius:8px;background:rgba(255,255,255,0.03);}

#eventLog{max-height:180px;overflow-y:auto;font-size:11px;color:var(--dim);}

```

```
.log-entry{padding:5px 0;border-bottom:1px solid rgba(255,255,255,0.04);font-family:'Consolas',monospace;}
```

```
.log-entry:first-child{color:#cfe8ff;}
```

```
.info-line{font-size:12px;color:var(--dim);display:flex;justify-content:space-between;padding:4px 0;}
```

```
.info-line b{color:#fff;}
```

```
/* ===== SECTIONS BELOW ===== */
```

```
.section-title{font-size:20px;font-weight:800;color:#fff;margin:26px 0 14px;text-align:center;letter-spacing:1px;}
```

```
.section-title span{color:var(--cyan);}
```

```
.grid3{display:grid;grid-template-columns:1fr 1fr 1fr;gap:14px;}
```

```
@media (max-width:900px){.grid3{grid-template-columns:1fr;}}
```

```
.overview-card{padding:18px;}
```

```
.overview-card h4{margin:0 0 10px;color:var(--cyan);font-size:14px;}
```

```
.overview-card ul{margin:0;padding-left:18px;color:var(--dim);font-size:12.5px;line-height:1.8;}
```

```
.overview-card p{color:var(--dim);font-size:12.5px;line-height:1.7;margin:0;}
```

```
/* ARCHITECTURE DIAGRAM */
```

```
.arch-flow{display:flex;flex-direction:column;align-items:center;gap:4px;padding:22px;}
```

```
.arch-box{padding:10px 22px;border-radius:10px;background:rgba(0,229,255,0.08);border:1px solid rgba(0,229,255,0.3);color:#cdf3ff;font-weight:700;font-size:12.5px;text-align:center;letter-spacing:0.5px;}
```

```
.arch-arrow{color:var(--dim2);font-size:16px;margin:2px 0;}
```

```
.arch-agents{display:grid;grid-template-columns:repeat(4,1fr);gap:6px;padding:10px;border:1px solid rgba(255,255,255,0.15);border-radius:10px;background:rgba(255,255,255,0.03);margin-top:2px;}
```

```
.arch-agents span{background:rgba(59,130,246,0.15);border:1px solid rgba(59,130,246,0.35);border-radius:6px;padding:6px 4px;text-align:center;font-size:11px;color:#a9c8ff;font-weight:700;}
```

```
/* FLOW DIAGRAM */
```

```
.flow-steps{display:flex;flex-wrap:wrap;justify-content:center;gap:8px;padding:16px;}
```

```
.flow-step{padding:10px 16px;border-
radius:20px;background:rgba(139,92,246,0.1);border:1px solid
rgba(139,92,246,0.35);color:#e0d4ff;font-size:12px;font-weight:600;display:flex;align-
items:center;gap:6px;}
```

```
.flow-step .num{background:#8b5cf6;color:#fff;width:18px;height:18px;border-
radius:50%;display:flex;align-items:center;justify-content:center;font-size:10px;}
```

```
.flow-arrow-sm{color:var(--dim2);font-size:14px;}
```

```
footer{text-align:center;padding:26px 10px;color:var(--dim);font-size:12px;border-top:1px
solid var(--card-border);margin-top:30px;}
```

```
footer .foot-title{color:#fff;font-weight:700;margin-bottom:6px;}
```

```
footer .foot-tag{color:var(--cyan);font-size:11px;margin-top:8px;letter-spacing:0.5px;}
```

```
footer .foot-brand{margin-top:14px;font-size:10.5px;color:var(--dim2);}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div class="app">
```

```
<!-- ===== TOP BAR ===== -->
```

```
<div class="topbar">
```

```
<div class="brand">
```

```
<div class="logo">SR-MACS</div>
```

```
<div class="subb">Swarm Robotics Multi-Agent<br>Coordination System</div>
```

```
</div>
```

```
<div class="center-title">
```

```
<h1>SWARM ROBOTICS</h1>
```

```
<div class="sub">MULTI-AGENT COORDINATION SYSTEM</div>
```

```
</div>
```

```
<div id="systemStatus" class="status-badge paused">⏸ SYSTEM READY</div>
```

```
</div>
```

```
<!-- ===== TITLE / CONTROLS ===== -->
```

```
<div class="titlebar glass">
```

```
<h2>Swarm Robotics Based Multi-Agent Coordination System</h2>
```

```
<div class="small-sub">Autonomous Multi-Agent Coordination, Navigation & Task Allocation</div>
```

```
<div class="ctrl-buttons">
```

```
<button class="btn btn-start" id="btnStart">▶ START SIMULATION</button>
```

```
<button class="btn btn-pause" id="btnPause">⏸ PAUSE</button>
```

```
<button class="btn btn-reset" id="btnReset">🔄 RESET</button>
```

```
<button class="btn btn-random" id="btnRandom">🎲 RANDOMIZE</button>
```

```
<button class="btn btn-emergency" id="btnEmergency">⊖ EMERGENCY STOP</button>
```

```
<button class="btn btn-export" id="btnExport">↓ EXPORT SIMULATION DATA</button>
```

```
</div>
```

```
</div>
```

```
<div class="emergency-banner" id="emergencyBanner">🛑 EMERGENCY STOP ACTIVATED — ALL ROBOTS HALTED</div>
```

```
<!-- ===== MAIN LAYOUT ===== -->
```

```
<div class="main-layout">
```

```
<!-- LEFT SIDEBAR -->
```

```
<div class="sidebar">
```

```
<div class="panel glass">
```

```
<h3>Swarm Configuration</h3>
```

```
<div class="field">
```

```
<label>Number of Robots <span class="val" id="valRobots">8</span></label>
```

```
<input type="range" id="sliderRobots" min="3" max="20" value="8" step="1">
```

```
</div>
```

```
<div class="field">
```

```
<label>Number of Targets <span class="val" id="valTargets">3</span></label>
```

```
<input type="range" id="sliderTargets" min="1" max="8" value="3" step="1">
```

```
</div>
```

```
<div class="field">
```

```
<label>Simulation Speed <span class="val" id="valSpeed">1x</span></label>
```



```

        <input type="range" id="sliderSpeed" min="0" max="3" value="1" step="1">
    </div>
    <div class="field">
        <label>Communication Range <span class="val" id="valComm">150
px</span></label>
        <input type="range" id="sliderComm" min="50" max="250" value="150" step="10">
    </div>
    <div class="field">
        <label>Obstacle Density <span class="val" id="valObstacle">Medium</span></label>
        <input type="range" id="sliderObstacle" min="0" max="2" value="1" step="1">
    </div>
</div>

<div class="panel glass">
    <h3>Coordination Algorithm</h3>
    <select id="selectAlgorithm">
        <option value="flocking">Flocking</option>
        <option value="leaderfollower">Leader-Follower</option>
        <option value="potentialfield">Potential Field</option>
        <option value="consensus">Consensus</option>
        <option value="nearest">Nearest Target Assignment</option>
        <option value="cooperative">Cooperative Navigation</option>
    </select>
    <div class="algo-desc" id="algoDesc"><b>Flocking</b>Robots maintain separation,
alignment and cohesion while moving toward their assigned targets.</div>
</div>

<div class="panel glass">
    <h3>Formation Mode</h3>
    <select id="selectFormation">
        <option value="free">Free Swarm</option>
        <option value="line">Line Formation</option>
        <option value="v">V Formation</option>

```

```
<option value="circle">Circle Formation</option>
<option value="grid">Grid Formation</option>
<option value="leaderfollower">Leader-Follower</option>
</select>
</div>
```

```
<div class="panel glass">
  <h3>Flocking Weights</h3>

  <div class="field"><label>Separation <span class="val"
id="valWSep">1.4</span></label><input type="range" id="wSep" min="0" max="3"
value="1.4" step="0.1"></div>

  <div class="field"><label>Alignment <span class="val"
id="valWAlign">1.0</span></label><input type="range" id="wAlign" min="0" max="3"
value="1.0" step="0.1"></div>

  <div class="field"><label>Cohesion <span class="val"
id="valWCoh">0.9</span></label><input type="range" id="wCoh" min="0" max="3"
value="0.9" step="0.1"></div>

  <div class="field"><label>Target Attraction <span class="val"
id="valWTarget">1.3</span></label><input type="range" id="wTarget" min="0" max="3"
value="1.3" step="0.1"></div>

  <div class="field"><label>Obstacle Avoidance <span class="val"
id="valWObs">1.7</span></label><input type="range" id="wObs" min="0" max="3"
value="1.7" step="0.1"></div>

</div>
```

```
<div class="panel glass">
  <h3>System Indicators</h3>

  <div class="indicators">

    <div>□ Navigation</div>

    <div>□ Communication</div>

    <div>□ Collision Avoidance</div>

    <div>□ Task Allocation</div>

    <div id="indFormation">○ Formation Control</div>

    <div>□ Swarm Coordination</div>

  </div>
```

</div>

</div>

<!-- MAIN SIMULATION -->

<div class="sim-wrap">

<canvas id="simCanvas" width="1100" height="620"></canvas>

<div class="mini-status" id="miniStatus"></div>

<div class="help-box glass">

INTERACTION

Click → Add Obstacle

Shift + Click → Add Target

Robot Click → Robot Details

Double-Click Target → Remove Target

</div>

<div class="details-panel glass hidden" id="robotDetailsPanel"></div>

</div>

<!-- RIGHT PANEL -->

<div class="rightpanel">

<div class="panel glass">

<h3>Performance Metrics</h3>

<div class="metrics-grid">

<div class="metric-card cyan"><div class="mv" id="metricActive">8</div><div class="ml">Active Robots</div></div>

<div class="metric-card green"><div class="mv" id="metricCompleted">0</div><div class="ml">Targets Completed</div></div>

<div class="metric-card red"><div class="mv" id="metricCollisions">0</div><div class="ml">Collisions</div></div>

<div class="metric-card orange"><div class="mv" id="metricObstacles">12</div><div class="ml">Obstacles</div></div>

<div class="metric-card cyan"><div class="mv" id="metricCommLinks">0</div><div class="ml">Comm Links</div></div>

<div class="metric-card green"><div class="mv" id="metricEfficiency">90%</div><div class="ml">Swarm Efficiency</div></div>

```
<div class="metric-card cyan"><div class="mv" id="metricAvgSpeed">0 m/s</div><div class="ml">Average Speed</div></div>
```

```
<div class="metric-card orange"><div class="mv" id="metricCoverage">0%</div><div class="ml">Coverage</div></div>
```

```
</div>
```

```
<div class="row-stats" style="margin-top:10px;"><span>Collision Avoidance</span><b style="color:#7CFFB2;">ACTIVE</b></div>
```

```
</div>
```

```
<div class="panel glass">
```

```
<h3>Communication Network</h3>
```

```
<div class="info-line"><span>Connected Robots</span><b id="commConnected">0</b></div>
```

```
<div class="info-line"><span>Messages / sec</span><b id="commMsgRate">0</b></div>
```

```
<div class="info-line"><span>Avg. Comm. Distance</span><b id="commAvgDist">0 px</b></div>
```

```
<div class="info-line"><span>Network Connectivity</span><b id="commConnectivity">0%</b></div>
```

```
<div id="leaderPanel" class="info-line" style="display:none;"><span>👤 LEADER</span><b id="leaderValue">R1</b></div>
```

```
<div id="consensusPanel" class="info-line" style="display:none;"><span>□ CONSENSUS LEVEL</span><b id="consensusValue">0%</b></div>
```

```
</div>
```

```
<div class="panel glass">
```

```
<h3>Task Allocation</h3>
```

```
<div class="table-scroll">
```

```
<table>
```

```
<thead><tr><th>Robot</th><th>Task</th><th>Distance</th><th>Status</th></tr></thead>
```

```
<tbody id="taskTableBody"></tbody>
```

```
</table>
```

```
</div>
```

```
</div>
```

```

<div class="panel glass">
  <h3>Robot Status Monitor</h3>
  <div class="table-scroll">
    <table>

<thead><tr><th>ID</th><th>X</th><th>Y</th><th>Spd</th><th>Tgt</th><th>Batt</th><th>
Status</th></tr></thead>

    <tbody id="robotTableBody"></tbody>
  </table>
</div>
</div>

```

```

<div class="panel glass">
  <h3>Real-Time Charts</h3>

  <div class="chart-box"><div class="clabel">Swarm Efficiency (%)</div><canvas
class="chart" id="chartEfficiency"></canvas></div>

  <div class="chart-box"><div class="clabel">Active Robots</div><canvas class="chart"
id="chartActive"></canvas></div>

  <div class="chart-box"><div class="clabel">Completed Tasks</div><canvas
class="chart" id="chartCompleted"></canvas></div>

  <div class="chart-box"><div class="clabel">Collision Count</div><canvas class="chart"
id="chartCollisions"></canvas></div>
</div>

```

```

<div class="panel glass">
  <h3>Event Log</h3>

  <div id="eventLog"></div>
</div>
</div>
</div>

```

```

<!-- ===== PROJECT OVERVIEW ===== -->
<div class="section-title">PROJECT <span>OVERVIEW</span></div>
<div class="grid3">

```

<div class="overview-card glass">

<h4>Objective</h4>

<p>To develop an autonomous multi-agent coordination system where multiple robots cooperate to navigate an environment, avoid obstacles, communicate, and complete assigned tasks efficiently.</p>

</div>

<div class="overview-card glass">

<h4>Key Technologies</h4>

- Multi-Agent Systems
- Swarm Intelligence
- Artificial Intelligence
- Autonomous Robotics
- Distributed Coordination
- Collision Avoidance
- Task Allocation
- HTML5 Canvas & JavaScript

</div>

<div class="overview-card glass">

<h4>Applications</h4>

- Search and Rescue
- Agriculture
- Military Surveillance
- Warehouse Automation
- Environmental Monitoring
- Disaster Management
- Smart Transportation
- Industrial Automation

</div>

</div>

<!-- ===== ARCHITECTURE ===== -->

<div class="section-title">SYSTEM ARCHITECTURE</div>

<div class="glass arch-flow">

<div class="arch-box">USER</div>

<div class="arch-arrow">↓</div>

<div class="arch-box">CONTROL PANEL</div>

<div class="arch-arrow">↓</div>

<div class="arch-box">SWARM COORDINATION ENGINE</div>

<div class="arch-arrow">↓</div>

<div class="arch-box">MULTI-AGENT DECISION MAKING</div>

<div class="arch-arrow">↓</div>

<div class="arch-agents">

R1R2R3R4

R5R6R7R8

</div>

<div class="arch-arrow">↓</div>

<div class="arch-box">COMMUNICATION NETWORK</div>

<div class="arch-arrow">↓</div>

<div class="arch-box">ENVIRONMENT</div>

<div class="arch-arrow">↓</div>

<div class="arch-box">TARGETS + OBSTACLES</div>

</div>

<!-- ===== ALGORITHM FLOW ===== -->

<div class="section-title">SWARM COORDINATION FLOW</div>

<div class="glass flow-steps">

<div class="flow-step">1Initialize Robots</div>→

<div class="flow-step">2Detect Environment</div>→

<div class="flow-step">3Detect Neighbours</div>→

```

    <div class="flow-step"><span class="num">4</span>Assign Targets</div><span
class="flow-arrow-sm">→</span>

    <div class="flow-step"><span class="num">5</span>Calculate Swarm
Forces</div><span class="flow-arrow-sm">→</span>

    <div class="flow-step"><span class="num">6</span>Avoid Obstacles</div><span
class="flow-arrow-sm">→</span>

    <div class="flow-step"><span class="num">7</span>Move Robots</div><span
class="flow-arrow-sm">→</span>

    <div class="flow-step"><span class="num">8</span>Check Target
Completion</div><span class="flow-arrow-sm">→</span>

    <div class="flow-step"><span class="num">9</span>Update Communication</div><span
class="flow-arrow-sm">→</span>

    <div class="flow-step"><span class="num">10</span>Update Metrics</div><span
class="flow-arrow-sm">→</span>

    <div class="flow-step"><span class="num">∞</span>Repeat</div>
</div>

```

```

<footer>

```

```

    <div class="foot-title">Swarm Robotics Based Multi-Agent Coordination System</div>

```

```

    <div>Autonomous Coordination • Intelligent Navigation • Distributed Decision
Making</div>

```

```

    <div class="foot-tag">AI & ROBOTICS SIMULATION PLATFORM</div>

```

```

    <div class="foot-brand">FINAL YEAR CAPSTONE PROJECT — SR-MACS</div>

```

```

</footer>

```

```

</div>

```

```

<script>

```

```

(function(){

```

```

"use strict";

```

```

/* ===== CONSTANTS ===== */

```

```

var WIDTH = 1100, HEIGHT = 620;

```

```

var POS_MULT = 1.5, MAX_SPEED = 2.2, MAX_SPEED_LOW = 1.1;

```

```

var COLLISION_DIST = 18, SEP_RADIUS = 45, TARGET_HIT = 16, TARGET_NEAR = 90;

```



```

var ALGO_DESC = {
  flocking:{name:"Flocking", text:"Robots maintain separation, alignment and cohesion while
moving toward their assigned targets."},

  leaderfollower:{name:"Leader-Follower", text:"One robot is designated leader and moves
toward the target. Other robots follow while keeping a relative formation offset."},

  potentialfield:{name:"Potential Field", text:"Targets act as attractive potentials and
obstacles/robots as repulsive potentials; robots move along the combined force field."},

  consensus:{name:"Consensus", text:"Robots exchange velocity information with
neighbours and gradually converge toward a shared heading and swarm state."},

  nearest:{name:"Nearest Target Assignment", text:"Each robot greedily selects and moves
directly toward its nearest available target using only local collision avoidance."},

  cooperative:{name:"Cooperative Navigation", text:"Robots communicate to divide tasks by
distance and workload, balancing flocking behaviour with obstacle-aware navigation."}
};

```

```

/* ===== STATE ===== */

```

```

var state = {
  robots:[], targets:[], obstacles:[], commEdges:[],
  algorithm:"flocking", formation:"free",
  config:{numRobots:8, numTargets:3, obstacleDensity:"medium"},
  weights:{sep:1.4, align:1.0, coh:0.9, target:1.3, obstacle:1.7},
  commRange:150, speed:1,
  running:false, emergency:false,
  metrics:{collisions:0, completed:0},
  charts:{efficiency:[], active:[], completed:[], collisions:[]},
  coverageGrid:[], targetCounter:1,
  leaderId:null, selectedRobotId:null,
  eventLog:[], startTime:null,
  formationCenter:{x:WIDTH/2, y:HEIGHT/2}
};

```

```

/* ===== UTIL ===== */

```

```

function rand(a,b){return a+Math.random()*(b-a);}

```

```

function dist(a,b){return Math.hypot(a.x-b.x,a.y-b.y);}

function randPos(margin){ margin=margin||60; return {x:rand(margin,WIDTH-margin),
y:rand(margin,HEIGHT-margin)}; }

function safeRandomPosition(){
    for(var i=0;i<30;i++){
        var p = randPos(60); var ok=true;
        for(var j=0;j<state.obstacles.length;j++){ var o=state.obstacles[j]; if(dist(p,o) < o.r+42){
ok=false; break; } }
        if(ok) return p;
    }
    return randPos(60);
}

function getRobotById(id){ for(var i=0;i<state.robots.length;i++){ if(state.robots[i].id===id)
return state.robots[i]; } return null; }

function getAssignedTarget(r){ if(!r.targetId) return null; for(var i=0;i<state.targets.length;i++){
if(state.targets[i].id===r.targetId) return state.targets[i]; } return null; }

function normalizeVec(v){ var m=Math.hypot(v.x,v.y); if(m>0){ v.x/=m; v.y/=m; } return v; }

function escapeHtml(s){ return
s.replace(/&/g,"&amp;").replace(/</g,"&lt;").replace(/>/g,"&gt;"); }

/* ===== EVENT LOG ===== */

function logEvent(msg){
    var t = new Date().toLocaleTimeString("en-GB",{hour12:false});
    state.eventLog.unshift("[ "+t+" ] "+msg);
    if(state.eventLog.length>50) state.eventLog.pop();
    renderEventLog();
}

function renderEventLog(){
    var el = document.getElementById("eventLog");
    var html = "";
    for(var i=0;i<state.eventLog.length;i++){ html += '<div class="log-
entry">'+escapeHtml(state.eventLog[i])+'</div>'; }
    el.innerHTML = html;
}

```

```

/* ===== INITIALIZATION ===== */
function regenerateObstacles(){
    state.obstacles = [];
    var densityMap = {low:6, medium:12, high:20};
    var count = densityMap[state.config.obstacleDensity];
    for(var i=0;i<count;i++){
        var p = randPos(50);
        state.obstacles.push({x:p.x, y:p.y, r: 14+Math.random()*16});
    }
}

function initSimulation(){
    regenerateObstacles();
    state.targets = [];
    state.targetCounter = 1;
    for(var i=0;i<state.config.numTargets;i++){
        var p = safeRandomPosition();
        state.targets.push({id:"T"+(state.targetCounter++), x:p.x, y:p.y, status:"WAITING"});
    }
    state.robots = [];
    for(var k=1;k<=state.config.numRobots;k++){
        var pr = safeRandomPosition();
        state.robots.push({
            id:"R"+k, x:pr.x, y:pr.y,
            vx:(Math.random()-0.5)*1.5, vy:(Math.random()-0.5)*1.5,
            battery: 85+Math.random()*15, status:"ACTIVE", targetId:null, collisionFlash:0
        });
    }
    state.leaderId = state.robots.length ? state.robots[0].id : null;
    state.metrics = {collisions:0, completed:0};
    state.coverageGrid = new Array(22*13).fill(false);
}

```

```

state.charts = {efficiency:[], active:[], completed:[], collisions:[]};
state.commEdges = [];
state.selectedRobotId = null;
document.getElementById("robotDetailsPanel").classList.add("hidden");
state.eventLog = [];
state.startTime = performance.now();
logEvent("SYSTEM INITIALIZED");
logEvent(state.robots.length+" robots deployed, "+state.targets.length+" targets,
"+state.obstacles.length+" obstacles");
assignTargets();
refreshPanels();
render();
}

/* ===== DYNAMIC ADJUSTMENTS ===== */
function adjustRobotCount(n){
    var cur = state.robots.length;
    if(n>cur){
        for(var i=cur+1;i<=n;i++){
            var p = safeRandomPosition();
            state.robots.push({id:"R"+i, x:p.x, y:p.y, vx:(Math.random()-0.5)*1.5, vy:(Math.random()-
0.5)*1.5, battery:100, status:"ACTIVE", targetId:null, collisionFlash:0});
        }
    } else if(n<cur){
        state.robots.splice(n);

        if(state.leaderId && !getRobotById(state.leaderId) && state.robots.length) state.leaderId =
state.robots[0].id;
    }
    assignTargets();
    logEvent("Swarm size adjusted to "+n+" robots");
}

function adjustTargetCount(n){
    var cur = state.targets.length;

```

```

if(n>cur){
    for(var i=cur;i<n;i++){
        var p = safeRandomPosition();
        state.targets.push({id:"T"+(state.targetCounter++), x:p.x, y:p.y, status:"WAITING"});
    }
} else if(n<cur){
    state.targets.splice(n);
}
assignTargets();
logEvent("Target count adjusted to "+n);
}

```

/* ===== TASK ALLOCATION ===== */

```

function assignTargets(){
    var crowd = {};
    for(var i=0;i<state.targets.length;i++) crowd[state.targets[i].id]=0;
    for(var r=0;r<state.robots.length;r++){
        var rob = state.robots[r];
        var t = getAssignedTarget(rob);
        if(t && t.status!="COMPLETED") crowd[t.id] = (crowd[t.id]||0)+1;
    }
    for(var k=0;k<state.robots.length;k++){
        var robot = state.robots[k];
        if(robot.battery<=0) continue;
        var cur = getAssignedTarget(robot);
        if(cur && cur.status!="COMPLETED") continue;
        var best=null, bestCost=Infinity;
        for(var j=0;j<state.targets.length;j++){
            var tg = state.targets[j];
            if(tg.status=="COMPLETED") continue;
            var d = dist(tg, robot);
            var penalty = (state.algorithm=="nearest") ? 0 : (crowd[tg.id]||0)*40;

```

```

    var cost = d+penalty;
    if(cost<bestCost){ bestCost=cost; best=tg; }
}
if(best){
    robot.targetId = best.id;
    crowd[best.id] = (crowd[best.id]||0)+1;
    if(best.status==="WAITING") best.status="ASSIGNED";
    logEvent(robot.id+" assigned to "+best.id);
}
}
}

/* ===== FORMATION LOGIC ===== */
function updateFormationCenter(){
    if(state.robots.length===0) return;
    var cx=0, cy=0;
    for(var i=0;i<state.robots.length;i++){ cx+=state.robots[i].x; cy+=state.robots[i].y; }
    cx/=state.robots.length; cy/=state.robots.length;
    state.formationCenter.x += (cx-state.formationCenter.x)*0.02;
    state.formationCenter.y += (cy-state.formationCenter.y)*0.02;
}

function getFollowerOffset(r){
    var leader = getRobotById(state.leaderId);
    if(!leader) return {x:0,y:0};
    var followers = state.robots.filter(function(x){ return x.id!==state.leaderId; });
    var idx = followers.indexOf(r);
    var col = (idx%3)-1, row = Math.floor(idx/3)+1;
    var ang = Math.atan2(leader.vy, leader.vx);
    var speed = Math.hypot(leader.vx,leader.vy);
    if(speed<0.05) ang = 0;
    var fx=Math.cos(ang), fy=Math.sin(ang);
    var rx=-fy, ry=fx;

```

```

    return { x: -fx*row*45 + rx*col*40, y: -fy*row*45 + ry*col*40 };
}

function getFormationSlot(r){
    var c = state.formationCenter;
    var idx = state.robots.indexOf(r);
    var n = state.robots.length;
    if(state.formation==="line"){
        return {x: c.x + (idx-(n-1)/2)*40, y: c.y};
    } else if(state.formation==="v"){
        if(idx===0) return {x:c.x, y:c.y};
        var rank = Math.ceil(idx/2), side = (idx%2===1)?1:-1;
        return {x: c.x - rank*35, y: c.y + side*rank*25};
    } else if(state.formation==="circle"){
        var angle = (idx/Math.max(n,1))*Math.PI*2;
        return {x: c.x+Math.cos(angle)*120, y: c.y+Math.sin(angle)*120};
    } else if(state.formation==="grid"){
        var cols = Math.ceil(Math.sqrt(n));
        var col = idx%cols, row = Math.floor(idx/cols);
        return {x: c.x+(col-(cols-1)/2)*45, y: c.y+(row-1)*45};
    } else if(state.formation==="leaderfollower"){
        if(r.id===state.leaderId) return null;
        var leader = getRobotById(state.leaderId);
        if(!leader) return null;
        var off = getFollowerOffset(r);
        return {x: leader.x+off.x, y: leader.y+off.y};
    }
    return null;
}

/* ===== FORCE CALCULATION ===== */

function computeForces(r){
    var sep={x:0,y:0}, align={x:0,y:0}, coh={x:0,y:0}, tgt={x:0,y:0}, obs={x:0,y:0},
    bound={x:0,y:0}, form={x:0,y:0};

```

```

var neighborCount=0;
for(var i=0;i<state.robots.length;i++){
    var other = state.robots[i];
    if(other===r) continue;
    var dx=r.x-other.x, dy=r.y-other.y, d=Math.hypot(dx,dy);
    if(d<state.commRange && d>0){
        neighborCount++;
        align.x+=other.vx; align.y+=other.vy;
        coh.x+=other.x; coh.y+=other.y;
    }
    if(d<SEP_RADIUS && d>0){
        var f=(SEP_RADIUS-d)/SEP_RADIUS;
        sep.x += (dx/d)*f; sep.y += (dy/d)*f;
    }
}
if(neighborCount>0){
    align.x/=neighborCount; align.y/=neighborCount;
    align.x-=r.vx; align.y-=r.vy;
    coh.x = coh.x/neighborCount - r.x;
    coh.y = coh.y/neighborCount - r.y;
    normalizeVec(coh);
}
normalizeVec(sep); normalizeVec(align);

var t = getAssignedTarget(r);
if(t){ tgt.x=t.x-r.x; tgt.y=t.y-r.y; normalizeVec(tgt); }

for(var o=0;o<state.obstacles.length;o++){
    var ob = state.obstacles[o];
    var odx=r.x-ob.x, ody=r.y-ob.y, od=Math.hypot(odx,ody);
    var threat = ob.r+35;
    if(od<threat && od>0){

```



```

    var of=(threat-od)/threat;
    obs.x += (odx/od)*of*2; obs.y += (ody/od)*of*2;
  }
}

```

```

var margin=40;
if(r.x<margin) bound.x += (margin-r.x)/margin;
if(r.x>WIDTH-margin) bound.x -= (r.x-(WIDTH-margin))/margin;
if(r.y<margin) bound.y += (margin-r.y)/margin;
if(r.y>HEIGHT-margin) bound.y -= (r.y-(HEIGHT-margin))/margin;

```

```

if(state.formation!="free"){
  var desired = getFormationSlot(r);
  if(desired){ form.x = desired.x-r.x; form.y = desired.y-r.y; normalizeVec(form); }
}

```

```

return {sep:sep, align:align, coh:coh, tgt:tgt, obs:obs, bound:bound, form:form,
neighborCount:neighborCount};
}

```

```

function applyAlgorithm(r, f){
  var w = state.weights;
  var ax=0, ay=0;
  switch(state.algorithm){
    case "flocking":
      ax = f.sep.x*w.sep + f.align.x*w.align + f.coh.x*w.coh + f.tgt.x*w.target +
f.obs.x*w.obstacle;
      ay = f.sep.y*w.sep + f.align.y*w.align + f.coh.y*w.coh + f.tgt.y*w.target +
f.obs.y*w.obstacle;
      break;
    case "leaderfollower":
      if(r.id===state.leaderId){
        ax = f.tgt.x*1.6 + f.obs.x*w.obstacle + f.sep.x*w.sep;

```

```

    ay = f.tgt.y*1.6 + f.obs.y*w.obstacle + f.sep.y*w.sep;
} else {
    var leader = getRobotByld(state.leaderId);
    if(leader){
        var off = getFollowerOffset(r);
        var dx=(leader.x+off.x)-r.x, dy=(leader.y+off.y)-r.y;
        var dn={x:dx,y:dy}; normalizeVec(dn);
        ax = dn.x*1.8 + f.sep.x*w.sep*1.2 + f.obs.x*w.obstacle;
        ay = dn.y*1.8 + f.sep.y*w.sep*1.2 + f.obs.y*w.obstacle;
    } else {
        ax = f.tgt.x*w.target; ay = f.tgt.y*w.target;
    }
}
break;
case "potentialfield":
    ax = f.tgt.x*w.target*1.3 + f.obs.x*w.obstacle*1.6 + f.sep.x*w.sep*1.4;
    ay = f.tgt.y*w.target*1.3 + f.obs.y*w.obstacle*1.6 + f.sep.y*w.sep*1.4;
    break;
case "consensus":
    ax = f.align.x*2.2 + f.coh.x*0.8 + f.tgt.x*0.5 + f.sep.x*w.sep + f.obs.x*w.obstacle;
    ay = f.align.y*2.2 + f.coh.y*0.8 + f.tgt.y*0.5 + f.sep.y*w.sep + f.obs.y*w.obstacle;
    break;
case "nearest":
    ax = f.tgt.x*2.0 + f.sep.x*w.sep*1.5 + f.obs.x*w.obstacle;
    ay = f.tgt.y*2.0 + f.sep.y*w.sep*1.5 + f.obs.y*w.obstacle;
    break;
case "cooperative":
    ax = f.sep.x*w.sep + f.align.x*w.align*0.6 + f.coh.x*w.coh*0.6 + f.tgt.x*w.target*1.3 +
f.obs.x*w.obstacle;
    ay = f.sep.y*w.sep + f.align.y*w.align*0.6 + f.coh.y*w.coh*0.6 + f.tgt.y*w.target*1.3 +
f.obs.y*w.obstacle;
    break;
}

```

```

    ax += f.bound.x*3 + f.form.x*(state.formation!="free"?1.6:0);
    ay += f.bound.y*3 + f.form.y*(state.formation!="free"?1.6:0);
    return {ax:ax, ay:ay};
}

/* ===== PHYSICS UPDATE ===== */
function updateSimulation(dt){
    updateFormationCenter();
    computeCommunicationEdges();

    for(var i=0;i<state.robots.length;i++){
        var r = state.robots[i];
        if(r.battery<=0){ r.vx=0; r.vy=0; continue; }
        var f = computeForces(r);
        var a = applyAlgorithm(r, f);
        r.vx = r.vx*0.88 + a.ax*0.12;
        r.vy = r.vy*0.88 + a.ay*0.12;
        var sp = Math.hypot(r.vx, r.vy);
        var maxSp = r.battery<20 ? MAX_SPEED_LOW : MAX_SPEED;
        if(sp>maxSp){ r.vx = r.vx/sp*maxSp; r.vy = r.vy/sp*maxSp; }
        r.x += r.vx*dt*POS_MULT;
        r.y += r.vy*dt*POS_MULT;
        r.x = Math.max(14, Math.min(WIDTH-14, r.x));
        r.y = Math.max(14, Math.min(HEIGHT-14, r.y));
        if(r.collisionFlash>0) r.collisionFlash = Math.max(0, r.collisionFlash-dt);
    }

    resolveObstacleCollisions();
    detectCollisions();
    checkTargetCompletion();
    updateBattery(dt);
    markCoverage();

```

```
}
```

```
function resolveObstacleCollisions(){  
  for(var i=0;i<state.robots.length;i++){  
    var r = state.robots[i];  
    for(var j=0;j<state.obstacles.length;j++){  
      var o = state.obstacles[j];  
      var dx=r.x-o.x, dy=r.y-o.y, d=Math.hypot(dx,dy);  
      var minD=o.r+10;  
      if(d<minD && d>0){  
        var overlap = minD-d;  
        r.x += (dx/d)*overlap;  
        r.y += (dy/d)*overlap;  
      }  
    }  
  }  
}
```

```
function detectCollisions(){  
  var n = state.robots.length;  
  for(var i=0;i<n;i++){  
    for(var j=i+1;j<n;j++){  
      var a = state.robots[i], b = state.robots[j];  
      if(a.battery<=0 || b.battery<=0) continue;  
      var dx=b.x-a.x, dy=b.y-a.y, d=Math.hypot(dx,dy);  
      if(d<COLLISION_DIST){  
        var overlap = (COLLISION_DIST-d)||0.01;  
        var nx = d>0?dx/d:1, ny = d>0?dy/d:0;  
        a.x -= nx*overlap*0.5; a.y -= ny*overlap*0.5;  
        b.x += nx*overlap*0.5; b.y += ny*overlap*0.5;  
        if(a.collisionFlash<=0 && b.collisionFlash<=0){  
          state.metrics.collisions++;  
        }  
      }  
    }  
  }  
}
```

```

        logEvent("Collision detected between "+a.id+" and "+b.id);
    }
    a.collisionFlash = 1.2; b.collisionFlash = 1.2;
    a.vx -= nx*0.5; a.vy -= ny*0.5;
    b.vx += nx*0.5; b.vy += ny*0.5;
}
}
}
}

```

```

function checkTargetCompletion(){
    for(var i=0;i<state.targets.length;i++){
        var t = state.targets[i];
        if(t.status==="COMPLETED") continue;
        var minDist = Infinity;
        for(var j=0;j<state.robots.length;j++){
            var r = state.robots[j];
            if(r.targetId===t.id){
                var d = dist(t,r);
                if(d<minDist) minDist=d;
                if(d<TARGET_HIT){
                    t.status="COMPLETED";
                    t.completedAt = performance.now();
                    state.metrics.completed++;
                    logEvent(t.id+" completed by "+r.id);
                    r.targetId=null;
                }
            }
        }
        if(t.status!=="COMPLETED"){
            t.status = minDist<TARGET_NEAR ? "IN PROGRESS" :
            (t.status==="WAITING"? "WAITING": "ASSIGNED");
        }
    }
}

```

```

}
var kept = [];
for(var k=0;k<state.targets.length;k++){
    var tg = state.targets[k];
    if(tg.status==="COMPLETED" && performance.now()-tg.completedAt>1600) continue;
    kept.push(tg);
}
state.targets = kept;
while(state.targets.length < state.config.numTargets){
    var p = safeRandomPosition();
    state.targets.push({id:"T"+(state.targetCounter++), x:p.x, y:p.y, status:"WAITING"});
}
}

```

```

function updateBattery(dt){
    for(var i=0;i<state.robots.length;i++){
        var r = state.robots[i];
        if(r.battery>0){
            var sp = Math.hypot(r.vx,r.vy);
            r.battery -= dt*(0.02 + sp*0.01);
            if(r.battery<0) r.battery=0;
        }
        if(r.battery<=0){ r.status="OFFLINE"; r.vx=0; r.vy=0; }
        else if(r.collisionFlash>0.4){ r.status="COLLISION WARNING"; }
        else if(r.battery<20){ r.status="LOW BATTERY"; }
        else { r.status="ACTIVE"; }
    }
}

```

```

function markCoverage(){
    var cellsX=22, cellsY=13;
    for(var i=0;i<state.robots.length;i++){

```

```

    var r = state.robots[i];
    var cx = Math.min(cellsX-1, Math.max(0, Math.floor(r.x/WIDTH*cellsX)));
    var cy = Math.min(cellsY-1, Math.max(0, Math.floor(r.y/HEIGHT*cellsY)));
    state.coverageGrid[cy*cellsX+cx] = true;
  }
}

function computeCoverage(){
  var visited=0;
  for(var i=0;i<state.coverageGrid.length;i++){ if(state.coverageGrid[i]) visited++; }
  return Math.round(visited/state.coverageGrid.length*100);
}

/* ===== COMMUNICATION ===== */
function computeCommunicationEdges(){
  var edges=[];
  var n = state.robots.length;
  for(var i=0;i<n;i++){
    for(var j=i+1;j<n;j++){
      var a=state.robots[i], b=state.robots[j];
      var d = dist(a,b);
      if(d<=state.commRange) edges.push({a:a,b:b,d:d});
    }
  }
  state.commEdges = edges;
}

function computeConnectivity(){
  var n = state.robots.length;
  if(n===0) return 0;
  var parent = []; for(var i=0;i<n;i++) parent.push(i);
  function find(x){ while(parent[x]!==x){ x=parent[x]; } return x; }
  function union(a,b){ var ra=find(a), rb=find(b); if(ra!==rb) parent[ra]=rb; }
  for(var e=0;e<state.commEdges.length;e++){

```

```

    var edge = state.commEdges[e];
    var ia = state.robots.indexOf(edge.a), ib = state.robots.indexOf(edge.b);
    union(ia,ib);
  }
  var counts={};
  for(var k=0;k<n;k++){ var root=find(k); counts[root]=(counts[root]||0)+1; }
  var maxComp = 0;
  for(var key in counts){ if(counts[key]>maxComp) maxComp=counts[key]; }
  return maxComp/n*100;
}

function reassignLeaderIfNeeded(){
  var leader = getRobotById(state.leaderId);
  if(!leader || leader.battery<=0){
    var candidates = state.robots.filter(function(r){ return r.battery>0; }).sort(function(a,b){
    return b.battery-a.battery; });
    if(candidates.length && candidates[0].id!=state.leaderId){
      state.leaderId = candidates[0].id;
      logEvent(candidates[0].id+" promoted to swarm leader");
    }
  }
}

/* ===== METRICS ===== */
function computeEfficiency(){
  var base = 90;
  base += state.metrics.completed*1.5;
  base -= state.metrics.collisions*2.5;
  var lowBatt = state.robots.filter(function(r){ return r.battery<20; }).length;
  base -= lowBatt*1.5;
  return Math.max(35, Math.min(99, Math.round(base)));
}

```



```

/* ===== UI REFRESH ===== */

function refreshPanels(){
    var activeRobots = state.robots.filter(function(r){ return r.battery>0; }).length;

    var edges = state.commEdges.length;

    var avgDist = edges ? (state.commEdges.reduce(function(s,e){ return s+e.d;
    },0)/edges).toFixed(0) : 0;

    reassignLeaderIfNeeded();

    var connectivity = computeConnectivity().toFixed(0);

    var messagesPerSec = Math.round(edges*3.2 + Math.random()*4);

    document.getElementById("metricActive").textContent = activeRobots;
    document.getElementById("metricCompleted").textContent = state.metrics.completed;
    document.getElementById("metricCollisions").textContent = state.metrics.collisions;
    document.getElementById("metricObstacles").textContent = state.obstacles.length;
    document.getElementById("metricCommLinks").textContent = edges;
    document.getElementById("metricEfficiency").textContent = computeEfficiency()+"%";

    var avgSpeed = state.robots.length ? (state.robots.reduce(function(s,r){ return
s+Math.hypot(r.vx,r.vy); },0)/state.robots.length*1.1).toFixed(1) : "0.0";

    document.getElementById("metricAvgSpeed").textContent = avgSpeed+" m/s";
    document.getElementById("metricCoverage").textContent = computeCoverage()+"%";

    document.getElementById("commConnected").textContent = state.robots.filter(function(r){
        return state.commEdges.some(function(e){ return e.a===r || e.b===r; });
    }).length;

    document.getElementById("commMsgRate").textContent = messagesPerSec;
    document.getElementById("commAvgDist").textContent = avgDist+" px";
    document.getElementById("commConnectivity").textContent = connectivity+"%";

    var leaderPanel = document.getElementById("leaderPanel");
    if(state.algorithm==="leaderfollower" || state.formation==="leaderfollower"){
        leaderPanel.style.display="flex";

        document.getElementById("leaderValue").textContent = state.leaderId || "-";
    } else { leaderPanel.style.display="none"; }
}

```

```

var consensusPanel = document.getElementById("consensusPanel");
if(state.algorithm==="consensus"){
  consensusPanel.style.display="flex";
  var sx=0, sy=0;
  for(var i=0;i<state.robots.length;i++){
    var r = state.robots[i];
    var s = Math.hypot(r.vx,r.vy)||0.001;
    sx += r.vx/s; sy += r.vy/s;
  }
  var mag = state.robots.length ? Math.hypot(sx,sy)/state.robots.length : 0;
  document.getElementById("consensusValue").textContent =
  Math.round(Math.min(100,mag*100))+ "%";
} else { consensusPanel.style.display="none"; }

```

```

document.getElementById("indFormation").innerHTML = (state.formation!=="free" ? "□" :
"○") + " Formation Control";

```

```

renderTaskTable();
renderRobotTable();
updateRobotDetailsPanel();
renderMiniStatus();
}

```

```

function renderMiniStatus(){
  var el = document.getElementById("miniStatus");
  var html = "";
  var completed = state.targets.filter(function(t){ return t.status==="COMPLETED"; }).length;
  html += '<span style="color:#7CFFB2;">🌀 '+ (state.config.numTargets)+ '
TARGETS</span>';
  html += '<span style="color:#a9c8ff;">🔗 '+state.commEdges.length+' LINKS</span>';
  if(state.formation==="leaderfollower" || state.algorithm==="leaderfollower") html += '<span
style="color:#ffd54a;">👤 LEADER: '+ (state.leaderId||"-")+ '</span>';

```

```

    el.innerHTML = html;
}

function renderTaskTable(){
    var tbody = document.getElementById("taskTableBody");
    var html = "";
    for(var i=0;i<state.robots.length;i++){
        var r = state.robots[i];
        var t = getAssignedTarget(r);
        var d = t ? dist(t,r).toFixed(0) : "-";
        var statusLabel = r.battery<=0 ? "OFFLINE" : (t ? (parseFloat(d)<20 ? "ARRIVING" :
"MOVING") : "IDLE");
        var cls = statusLabel==="ARRIVING" ? "arriving" : (statusLabel==="MOVING" ? "moving"
: "idle");
        html += "<tr><td>"+r.id+"</td><td>"+(t?t.id:"—")+"</td><td>"+(t?d+" px":"—
")+"</td><td><span class='tag "+cls+"'>"+statusLabel+"</span></td></tr>";
    }
    tbody.innerHTML = html;
}

```

```

function renderRobotTable(){
    var tbody = document.getElementById("robotTableBody");
    var html = "";
    var map = {"ACTIVE":"ok","LOW BATTERY":"warn","COLLISION
WARNING":"danger","OFFLINE":"off"};
    for(var i=0;i<state.robots.length;i++){
        var r = state.robots[i];
        var speed = Math.hypot(r.vx,r.vy).toFixed(1);
        var t = getAssignedTarget(r);
        var battClass = r.battery<20 ? "batt-low" : (r.battery<50 ? "batt-mid" : "batt-ok");
        var badgeCls = map[r.status] || "ok";
        var statusText = r.status==="LOW BATTERY" ? "⚠ LOW BATTERY" : r.status;
        html +=
"<tr><td>"+r.id+"</td><td>"+Math.round(r.x)+"</td><td>"+Math.round(r.y)+"</td><td>"+spee

```

```

d+"</td><td>" + (t ? t.id : "-") + "</td><td>
class="" + battClass + "" + ">" + Math.round(r.battery) + "%</td><td><span class='badge
"+badgeCls+">" + statusText + "</span></td></tr>";

```

```

}

```

```

tbody.innerHTML = html;

```

```

}

```

```

function updateRobotDetailsPanel(){

```

```

    if(!state.selectedRobotId) return;

```

```

    var r = getRobotById(state.selectedRobotId);

```

```

    var panel = document.getElementById("robotDetailsPanel");

```

```

    if(!r){ panel.classList.add("hidden"); return; }

```

```

    var speed = (Math.hypot(r.vx,r.vy)*1.1).toFixed(2);

```

```

    var dir = (Math.atan2(r.vy,r.vx)*180/Math.PI).toFixed(0);

```

```

    var t = getAssignedTarget(r);

```

```

    var distToTarget = t ? dist(t,r).toFixed(0) : "-";

```

```

    var neighbors = state.robots.filter(function(o){ return o!==r && dist(o,r)<state.commRange;
}).length;

```

```

    panel.innerHTML =

```

```

        '<div class="panel-head"><strong>ROBOT '+r.id+'</strong><button class="close-btn"
onclick="window.__closeRobotDetails()">X</button></div>'+

```

```

        '<div class="detail-row"><span>Position</span><b>X='+Math.round(r.x)+'',
Y='+Math.round(r.y)+'</b></div>'+

```

```

        '<div class="detail-row"><span>Velocity</span><b>'+speed+' m/s</b></div>'+

```

```

        '<div class="detail-row"><span>Direction</span><b>'+dir+'°</b></div>'+

```

```

        '<div class="detail-
row"><span>Battery</span><b>'+Math.round(r.battery)+'%</b></div>'+

```

```

        '<div class="detail-row"><span>Target</span><b>'+(t?t.id:"None")+'</b></div>'+

```

```

        '<div class="detail-row"><span>Dist. to Target</span><b>'+distToTarget+' px</b></div>'+

```

```

        '<div class="detail-row"><span>Comm. Neighbours</span><b>'+neighbors+'</b></div>'+

```

```

        '<div class="detail-row"><span>Status</span><b>'+r.status+'</b></div>';

```

```

}

```

```

window.__closeRobotDetails = function(){

```

```

    state.selectedRobotId = null;

```

```

document.getElementById("robotDetailsPanel").classList.add("hidden");
};

/* ===== CHARTS ===== */
function drawChart(canvas, data, color){
    var ctx = canvas.getContext("2d");
    var w = canvas.width = canvas.clientWidth;
    var h = canvas.height = canvas.clientHeight;
    ctx.clearRect(0,0,w,h);
    ctx.strokeStyle = "rgba(255,255,255,0.06)";
    ctx.lineWidth = 1;

    for(var i=1;i<4;i++){ var y=h/4*i; ctx.beginPath(); ctx.moveTo(0,y); ctx.lineTo(w,y);
    ctx.stroke(); }

    if(data.length<2) return;
    var maxVal = Math.max.apply(null, data.concat([1]))*1.15;
    ctx.beginPath();
    for(var idx=0; idx<data.length; idx++){
        var x = (idx/(data.length-1))*w;
        var yy = h - (data[idx]/maxVal)*h;
        if(idx===0) ctx.moveTo(x,yy); else ctx.lineTo(x,yy);
    }
    ctx.strokeStyle = color; ctx.lineWidth = 2; ctx.stroke();
    ctx.lineTo(w,h); ctx.lineTo(0,h); ctx.closePath();
    var grad = ctx.createLinearGradient(0,0,0,h);
    grad.addColorStop(0, color+"55");
    grad.addColorStop(1, color+"00");
    ctx.fillStyle = grad; ctx.fill();
    var last = data[data.length-1];
    ctx.fillStyle = color; ctx.font = "bold 11px Segoe UI";
    ctx.fillText(Math.round(last*10)/10, w-30, 13);
}

function sampleCharts(){
    state.charts.encyclopedia.push(computeEfficiency());

```

```

state.charts.active.push(state.robots.filter(function(r){ return r.battery>0; }).length);
state.charts.completed.push(state.metrics.completed);
state.charts.collisions.push(state.metrics.collisions);
["efficiency","active","completed","collisions"].forEach(function(k){
    if(state.charts[k].length>40) state.charts[k].shift();
});
drawChart(document.getElementById("chartEfficiency"), state.charts.afficiency, "#00e5ff");
drawChart(document.getElementById("chartActive"), state.charts.active, "#22c55e");
drawChart(document.getElementById("chartCompleted"), state.charts.completed,
"##ff9800");
drawChart(document.getElementById("chartCollisions"), state.charts.collisions, "#ff3b3b");
}

/* ===== RENDERING ===== */
var canvas = document.getElementById("simCanvas");
var ctx = canvas.getContext("2d");

function drawBackgroundGrid(){
    ctx.strokeStyle = "rgba(255,255,255,0.035)";
    ctx.lineWidth = 1;

    for(var x=0;x<WIDTH;x+=40){ ctx.beginPath(); ctx.moveTo(x,0); ctx.lineTo(x,HEIGHT);
    ctx.stroke(); }

    for(var y=0;y<HEIGHT;y+=40){ ctx.beginPath(); ctx.moveTo(0,y); ctx.lineTo(WIDTH,y);
    ctx.stroke(); }
}

function drawObstacles(){
    for(var i=0;i<state.obstacles.length;i++){
        var o = state.obstacles[i];
        var grad = ctx.createRadialGradient(o.x,o.y,0,o.x,o.y,o.r);
        grad.addColorStop(0,"rgba(255,90,90,0.35)");
        grad.addColorStop(1,"rgba(120,20,20,0.15)");
        ctx.beginPath(); ctx.arc(o.x,o.y,o.r+8,0,Math.PI*2); ctx.fillStyle=grad; ctx.fill();
        ctx.beginPath(); ctx.arc(o.x,o.y,o.r,0,Math.PI*2);
    }
}

```

```

    ctx.fillStyle = "#3a1010"; ctx.fill();
    ctx.strokeStyle = "rgba(255,120,120,0.6)"; ctx.lineWidth=1.5; ctx.stroke();
  }
}

function drawCommLinks(){
  var now = performance.now();
  for(var i=0;i<state.commEdges.length;i++){
    var e = state.commEdges[i];
    var alpha = Math.max(0.08, 1-(e.d/state.commRange));
    ctx.beginPath();
    ctx.setLineDash([6,4]);
    ctx.lineDashOffset = -(now/50)%10;
    ctx.moveTo(e.a.x,e.a.y); ctx.lineTo(e.b.x,e.b.y);
    ctx.strokeStyle = "rgba(0,229,255,"+alpha.toFixed(2)+")";
    ctx.lineWidth = 1.4;
    ctx.stroke();
    ctx.setLineDash([]);
  }
}

function drawTargets(){
  var now = performance.now();

  var colors = {WAITING:"#5f7291", ASSIGNED:"#ff9800", "IN PROGRESS": "#00e5ff",
    COMPLETED:"#22c55e"};

  for(var i=0;i<state.targets.length;i++){
    var t = state.targets[i];
    var pulse = 5+Math.sin(now/260+i)*3;
    var col = colors[t.status] || "#5f7291";
    ctx.beginPath(); ctx.arc(t.x,t.y,14+pulse,0,Math.PI*2);
    ctx.strokeStyle = col+"66"; ctx.lineWidth=2; ctx.stroke();
    ctx.beginPath(); ctx.arc(t.x,t.y,10,0,Math.PI*2);
    ctx.fillStyle = col+"33"; ctx.fill();
    ctx.strokeStyle = col; ctx.lineWidth=2; ctx.stroke();
    ctx.beginPath(); ctx.moveTo(t.x-6,t.y); ctx.lineTo(t.x+6,t.y);

```

```

ctx.moveTo(t.x,t.y-6); ctx.lineTo(t.x,t.y+6);
ctx.strokeStyle = col; ctx.lineWidth=1.5; ctx.stroke();
ctx.fillStyle = "#dce8fb"; ctx.font="10px Segoe UI"; ctx.textAlign="center";
ctx.fillText(t.id, t.x, t.y-22);
if(t.status==="COMPLETED"){
    ctx.fillStyle="#22c55e"; ctx.font="bold 9px Segoe UI";
    ctx.fillText("□ DONE", t.x, t.y+28);
}
ctx.textAlign="left";
}
}

function drawRobots(){
    for(var i=0;i<state.robots.length;i++){
        var r = state.robots[i];

        var isLeader = (r.id===state.leaderId) && (state.algorithm==="leaderfollower" ||
state.formation==="leaderfollower");

        var isSelected = r.id===state.selectedRobotId;
        var color = "#00e5ff";
        if(r.status==="LOW BATTERY") color="#ff9800";
        if(r.status==="OFFLINE") color="#5f7291";
        if(r.collisionFlash>0.4) color="#ff3b3b";

        if(isSelected){
            ctx.beginPath(); ctx.arc(r.x,r.y,18+Math.sin(performance.now()/150)*2,0,Math.PI*2);
            ctx.strokeStyle="rgba(0,229,255,0.8)"; ctx.lineWidth=2; ctx.stroke();
        }
        if(isLeader){
            ctx.beginPath(); ctx.arc(r.x,r.y,15,0,Math.PI*2);
            ctx.strokeStyle="#ffd54a"; ctx.lineWidth=2.2; ctx.stroke();
        }

        var ang = Math.atan2(r.vy,r.vx);
        ctx.beginPath(); ctx.arc(r.x,r.y,9,0,Math.PI*2);

```



```

var g = ctx.createRadialGradient(r.x-2,r.y-2,1,r.x,r.y,10);
g.addColorStop(0,"#ffffff"); g.addColorStop(0.25,color); g.addColorStop(1,"#0a1428");
ctx.fillStyle=g; ctx.fill();
ctx.strokeStyle=color; ctx.lineWidth=1.5; ctx.stroke();

ctx.beginPath();
ctx.moveTo(r.x+Math.cos(ang)*9, r.y+Math.sin(ang)*9);
ctx.lineTo(r.x+Math.cos(ang)*15, r.y+Math.sin(ang)*15);
ctx.strokeStyle = color; ctx.lineWidth=2; ctx.stroke();

ctx.fillStyle="#cfe8ff"; ctx.font="9px Segoe UI"; ctx.textAlign="center";
ctx.fillText(r.id, r.x, r.y-14);

var battWidth = 16;
ctx.fillStyle="rgba(0,0,0,0.5)"; ctx.fillRect(r.x-battWidth/2, r.y+12, battWidth, 3);
var bcol = r.battery<20 ? "#ff3b3b" : (r.battery<50 ? "#ff9800" : "#22c55e");
ctx.fillStyle=bcol; ctx.fillRect(r.x-battWidth/2, r.y+12, battWidth*(r.battery/100), 3);
ctx.textAlign="left";
}
}
function render(){
  ctx.clearRect(0,0,WIDTH,HEIGHT);
  drawBackgroundGrid();
  drawObstacles();
  drawCommLinks();
  drawTargets();
  drawRobots();
}

/* ===== CANVAS INTERACTION ===== */
function getCanvasCoords(evt){
  var rect = canvas.getBoundingClientRect();

```

```

var scaleX = WIDTH/rect.width, scaleY = HEIGHT/rect.height;
return { x:(evt.clientX-rect.left)*scaleX, y:(evt.clientY-rect.top)*scaleY };
}

function addObstacle(x,y){
    state.obstacles.push({x:x, y:y, r:14+Math.random()*14});
    logEvent("Obstacle added at (" +Math.round(x)+", "+Math.round(y)+")");
}

function addTarget(x,y){
    var t = {id:"T"+(state.targetCounter++), x:x, y:y, status:"WAITING"};
    state.targets.push(t);
    logEvent("New target "+t.id+" created by user");
    assignTargets();
}

function removeTargetAt(x,y){
    for(var i=0;i<state.targets.length;i++){
        var t = state.targets[i];
        if(Math.hypot(t.x-x,t.y-y)<20){
            state.targets.splice(i,1);
            logEvent("Target "+t.id+" removed by user");
            return true;
        }
    }
    return false;
}

var lastClickTime=0, lastClickPos=null;
canvas.addEventListener("click", function(e){
    var now = Date.now();
    var p = getCanvasCoords(e);
    if(now-lastClickTime<350 && lastClickPos && Math.hypot(p.x-lastClickPos.x,p.y-
lastClickPos.y)<20){
        removeTargetAt(p.x,p.y);
        lastClickTime=0;
    }

```

```

    render();
    return;
}
lastClickTime = now; lastClickPos = p;

var hit=null;
for(var i=0;i<state.robots.length;i++){
    if(Math.hypot(state.robots[i].x-p.x, state.robots[i].y-p.y)<16){ hit=state.robots[i]; break; }
}
if(hit){
    state.selectedRobotId = hit.id;
    document.getElementById("robotDetailsPanel").classList.remove("hidden");
    updateRobotDetailsPanel();
    render();
    return;
}
if(e.shiftKey){ addTarget(p.x,p.y); } else { addObstacle(p.x,p.y); }
render();
});

/* ===== CSV EXPORT ===== */
function exportCSV(){
    var rows = [["Time(s)", "RobotID", "X", "Y", "Velocity", "Target", "Battery", "Status"]];
    var t = state.startTime ? ((performance.now()-state.startTime)/1000).toFixed(1) : "0.0";
    for(var i=0;i<state.robots.length;i++){
        var r = state.robots[i];
        var speed = Math.hypot(r.vx,r.vy).toFixed(2);
        var tg = getAssignedTarget(r);
        rows.push([t, r.id, Math.round(r.x), Math.round(r.y), speed, tg?tg.id:"- ",
Math.round(r.battery), r.status]);
    }
    var csv = rows.map(function(row){ return row.join(","); }).join("\n");
    var blob = new Blob([csv], {type:"text/csv"});

```

```

var url = URL.createObjectURL(blob);
var a = document.createElement("a");
a.href=url; a.download="swarm_simulation_data.csv";
document.body.appendChild(a); a.click(); document.body.removeChild(a);
URL.revokeObjectURL(url);
logEvent("Simulation data exported to CSV");
}

/* ===== CONTROLS / BUTTONS ===== */
function updateSystemStatus(){
  var el = document.getElementById("systemStatus");
  el.className = "status-badge";

  if(state.emergency){ el.classList.add("emergency"); el.textContent="🚨 EMERGENCY
STOP ACTIVATED"; }

  else if(state.running){ el.textContent="✅ SYSTEM ONLINE"; }

  else { el.classList.add("paused"); el.textContent="⏸ SYSTEM PAUSED"; }
}

function startSim(){
  state.emergency=false;
  document.getElementById("emergencyBanner").classList.remove("show");
  state.running=true;
  if(!state.startTime) state.startTime=performance.now();
  logEvent("Swarm deployed — simulation started");
  updateSystemStatus();
}

function pauseSim(){
  state.running=false;
  logEvent("Simulation paused");
  updateSystemStatus();
}

function resetSim(){
  state.running=false;

```

```

state.emergency=false;
document.getElementById("emergencyBanner").classList.remove("show");
initSimulation();
updateSystemStatus();
}

function randomizeSim(){
    initSimulation();
    logEvent("Swarm configuration randomized");
}

function emergencyStopSim(){
    state.running=false;
    state.emergency=true;
    for(var i=0;i<state.robots.length;i++){ state.robots[i].vx=0; state.robots[i].vy=0; }
    document.getElementById("emergencyBanner").classList.add("show");
    logEvent("EMERGENCY STOP ACTIVATED — all robots halted");
    updateSystemStatus();
}

document.getElementById("btnStart").addEventListener("click", startSim);
document.getElementById("btnPause").addEventListener("click", pauseSim);
document.getElementById("btnReset").addEventListener("click", resetSim);
document.getElementById("btnRandom").addEventListener("click", randomizeSim);
document.getElementById("btnEmergency").addEventListener("click", emergencyStopSim);
document.getElementById("btnExport").addEventListener("click", exportCSV);

/* Sliders */

var sliderRobots = document.getElementById("sliderRobots"), valRobots =
document.getElementById("valRobots");

sliderRobots.addEventListener("input", function(){
    valRobots.textContent=sliderRobots.value; });

sliderRobots.addEventListener("change", function(){
    state.config.numRobots=parseInt(sliderRobots.value,10);
    adjustRobotCount(state.config.numRobots); });

```

```
var sliderTargets = document.getElementById("sliderTargets"), valTargets =  
document.getElementById("valTargets");
```

```
sliderTargets.addEventListener("input", function(){  
valTargets.textContent=sliderTargets.value; });
```

```
sliderTargets.addEventListener("change", function(){  
state.config.numTargets=parseInt(sliderTargets.value,10);  
adjustTargetCount(state.config.numTargets); });
```

```
var speedMap=[0.5,1,2,3];
```

```
var sliderSpeed = document.getElementById("sliderSpeed"), valSpeed =  
document.getElementById("valSpeed");
```

```
sliderSpeed.addEventListener("input", function(){  
    state.speed = speedMap[parseInt(sliderSpeed.value,10)];  
    valSpeed.textContent = state.speed+"x";  
});
```

```
var sliderComm = document.getElementById("sliderComm"), valComm =  
document.getElementById("valComm");
```

```
sliderComm.addEventListener("input", function(){  
    state.commRange = parseInt(sliderComm.value,10);  
    valComm.textContent = state.commRange+" px";  
});
```

```
var obstacleMap=["low","medium","high"], obstacleLabels=["Low","Medium","High"];
```

```
var sliderObstacle = document.getElementById("sliderObstacle"), valObstacle =  
document.getElementById("valObstacle");
```

```
sliderObstacle.addEventListener("input", function(){ valObstacle.textContent =  
obstacleLabels[parseInt(sliderObstacle.value,10)]; });
```

```
sliderObstacle.addEventListener("change", function(){  
    state.config.obstacleDensity = obstacleMap[parseInt(sliderObstacle.value,10)];  
    regenerateObstacles();  
    logEvent("Obstacle density set to  
"+obstacleLabels[parseInt(sliderObstacle.value,10)].toUpperCase()+"  
("+state.obstacles.length+" obstacles)");  
    render();  
});
```

```

var selectAlgorithm = document.getElementById("selectAlgorithm");
selectAlgorithm.addEventListener("change", function(){
    state.algorithm = selectAlgorithm.value;
    var d = ALGO_DESC[state.algorithm];
    document.getElementById("algoDesc").innerHTML = "<b>" + d.name + "</b>" + d.text;
    logEvent("Coordination algorithm changed to " + d.name);
    assignTargets();
});

```

```

var selectFormation = document.getElementById("selectFormation");

var formationLabels={free:"Free Swarm", line:"Line Formation", v:"V Formation",
circle:"Circle Formation", grid:"Grid Formation", leaderfollower:"Leader-Follower"};

selectFormation.addEventListener("change", function(){
    state.formation = selectFormation.value;

    logEvent("Formation mode set to " + formationLabels[state.formation]);
});

```

```

function bindWeightSlider(id, key, labelId){
    var el = document.getElementById(id), lab = document.getElementById(labelId);
    el.addEventListener("input", function(){
        state.weights[key] = parseFloat(el.value);
        lab.textContent = el.value;
    });
}

```

```

bindWeightSlider("wSep", "sep", "valWSep");
bindWeightSlider("wAlign", "align", "valWAlign");
bindWeightSlider("wCoh", "coh", "valWCoh");
bindWeightSlider("wTarget", "target", "valWTarget");
bindWeightSlider("wObs", "obstacle", "valWObs");

```

```

/* ===== MAIN LOOP ===== */

var lastTime = performance.now();

```

```

var frameCount = 0;
function animate(now){
    requestAnimationFrame(animate);
    var rawDt = Math.min((now-lastTime)/16.6667, 3);
    lastTime = now;
    if(state.running && !state.emergency){
        var dt = rawDt*state.speed;
        updateSimulation(dt);
    }
    frameCount++;
    if(frameCount % 15 === 0){
        assignTargets();
        refreshPanels();
    }
    if(frameCount % 30 === 0){
        sampleCharts();
    }
    render();
}

/* ===== BOOTSTRAP ===== */
initSimulation();
updateSystemStatus();
requestAnimationFrame(animate);

})();
</script>
</body>
</html>

```


Output:

